

```
In [2]: # Import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Set plot style for better visuals
sns.set_style('whitegrid')
```

```
In [3]: # Load the cleaned dataset from Project 1
df = pd.read_excel('Dataset for Analytics CLEANED.xlsx')

# Display first 5 rows
df.head()
```

```
Out[3]:
```

	OrderID	Date	CustomerID	Product	Quantity	UnitPrice	ShippingAddress
0	ORD200000	2023-01-04	C72649	Monitor	5	570.62	928 Main St
1	ORD200001	2024-08-23	C75739	Phone	2	151.35	823 Main St
2	ORD200002	2024-02-27	C81728	Tablet	5	550.68	512 Main St
3	ORD200003	2023-10-15	C33540	Chair	1	273.19	275 Main St
4	ORD200004	2025-05-08	C81840	Printer	4	626.01	668 Main St

```
In [4]: # Check dataset shape and info
print("=== Dataset Overview ===\n")
print(f"Total rows: {df.shape[0]}")
print(f"Total columns: {df.shape[1]}\n")

print("=== Column Data Types ===\n")
print(df.dtypes)
```

=== Dataset Overview ===

Total rows: 1200
Total columns: 14

=== Column Data Types ===

```
OrderID                str
Date                   datetime64[us]
CustomerID             str
Product                str
Quantity               int64
UnitPrice              float64
ShippingAddress         str
PaymentMethod           str
OrderStatus            str
TrackingNumber          str
ItemsInCart            int64
CouponCode             str
ReferralSource          str
TotalPrice             float64
dtype: object
```

```
In [5]: # Generate descriptive statistics for numeric columns
print("=== Descriptive Statistics (Mean, Median, Count, etc.) ===\n")
print(df[['Quantity', 'UnitPrice', 'TotalPrice', 'ItemsInCart']].describe)
```

=== Descriptive Statistics (Mean, Median, Count, etc.) ===

	Quantity	UnitPrice	TotalPrice	ItemsInCart
count	1200.000000	1200.000000	1200.000000	1200.000000
mean	2.945833	356.412750	1053.968300	5.485000
std	1.407557	197.177146	819.856558	2.281983
min	1.000000	11.390000	11.390000	1.000000
25%	2.000000	186.062500	410.520000	4.000000
50%	3.000000	364.210000	823.615000	5.000000
75%	4.000000	521.570000	1578.475000	7.000000
max	5.000000	699.930000	3456.400000	10.000000

```
In [6]: # Calculate mean and median for key numeric columns
numeric_cols = ['Quantity', 'UnitPrice', 'TotalPrice', 'ItemsInCart']

print("=== Mean vs Median Comparison ===\n")
for col in numeric_cols:
    mean_val = df[col].mean()
    median_val = df[col].median()
    print(f"{col}: Mean = {mean_val:.2f}, Median = {median_val:.2f}")
```

=== Mean vs Median Comparison ===

Quantity: Mean = 2.95, Median = 3.00
UnitPrice: Mean = 356.41, Median = 364.21
TotalPrice: Mean = 1053.97, Median = 823.62
ItemsInCart: Mean = 5.49, Median = 5.00

```
In [8]: # Count of orders by Product
print("=== Orders by Product ===\n")
print(df['Product'].value_counts())

print("\n=== Orders by OrderStatus ===\n")
print(df['OrderStatus'].value_counts())
```

```
print("\n=== Orders by PaymentMethod ===\n")
print(df['PaymentMethod'].value_counts())
```

=== Orders by Product ===

```
Product
Printer    181
Tablet     179
Chair      178
Laptop     173
Desk       170
Monitor    163
Phone      156
Name: count, dtype: int64
```

=== Orders by OrderStatus ===

```
OrderStatus
Cancelled    250
Returned     247
Pending      237
Shipped      235
Delivered    231
Name: count, dtype: int64
```

=== Orders by PaymentMethod ===

```
PaymentMethod
Online       258
Cash         246
Credit Card  234
Debit Card   232
Gift Card    230
Name: count, dtype: int64
```

```
In [7]: # Ensure Date is datetime
df['Date'] = pd.to_datetime(df['Date'])

# Group by month and sum TotalPrice
df['Month'] = df['Date'].dt.to_period('M')
monthly_sales = df.groupby('Month')['TotalPrice'].sum().reset_index()
monthly_sales['Month'] = monthly_sales['Month'].astype(str)

print("=== Monthly Sales Trend ===\n")
print(monthly_sales.head(10))
```

=== Monthly Sales Trend ===

	Month	TotalPrice
0	2023-01	56685.75
1	2023-02	40117.66
2	2023-03	48609.37
3	2023-04	27751.71
4	2023-05	63836.84
5	2023-06	49500.19
6	2023-07	42820.66
7	2023-08	54352.14
8	2023-09	29526.67
9	2023-10	52607.85

```
In [9]: # Group by Product and sum TotalPrice
product_revenue = df.groupby('Product')['TotalPrice'].sum().sort_values(ascending=False)

print("=== Top 5 Products by Revenue ===\n")
print(product_revenue.head(5))

print("\n=== Bottom 5 Products by Revenue ===\n")
print(product_revenue.tail(5))
```

=== Top 5 Products by Revenue ===

```
Product
Chair      195620.11
Printer    195612.61
Laptop     192126.56
Tablet     186568.95
Monitor    175651.41
Name: TotalPrice, dtype: float64
```

=== Bottom 5 Products by Revenue ===

```
Product
Laptop     192126.56
Tablet     186568.95
Monitor    175651.41
Desk       167459.93
Phone      151722.39
Name: TotalPrice, dtype: float64
```

```
In [10]: # Detect outliers in TotalPrice using IQR
Q1 = df['TotalPrice'].quantile(0.25)
Q3 = df['TotalPrice'].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers = df[(df['TotalPrice'] < lower_bound) | (df['TotalPrice'] > upper_bound)]

print("=== Outlier Detection (TotalPrice) ===\n")
print(f"Q1: {Q1:.2f}")
print(f"Q3: {Q3:.2f}")
print(f"IQR: {IQR:.2f}")
print(f"Lower Bound: {lower_bound:.2f}")
print(f"Upper Bound: {upper_bound:.2f}")
print(f"Number of Outliers: {len(outliers)}")
print(f"Percentage of Outliers: {len(outliers)/len(df)*100:.2f}%")
```

=== Outlier Detection (TotalPrice) ===

```
Q1: 410.52
Q3: 1578.47
IQR: 1167.95
Lower Bound: -1341.41
Upper Bound: 3330.41
Number of Outliers: 8
Percentage of Outliers: 0.67%
```

```
In [11]: # Select numeric columns for correlation
numeric_df = df[['Quantity', 'UnitPrice', 'TotalPrice', 'ItemsInCart']]
```

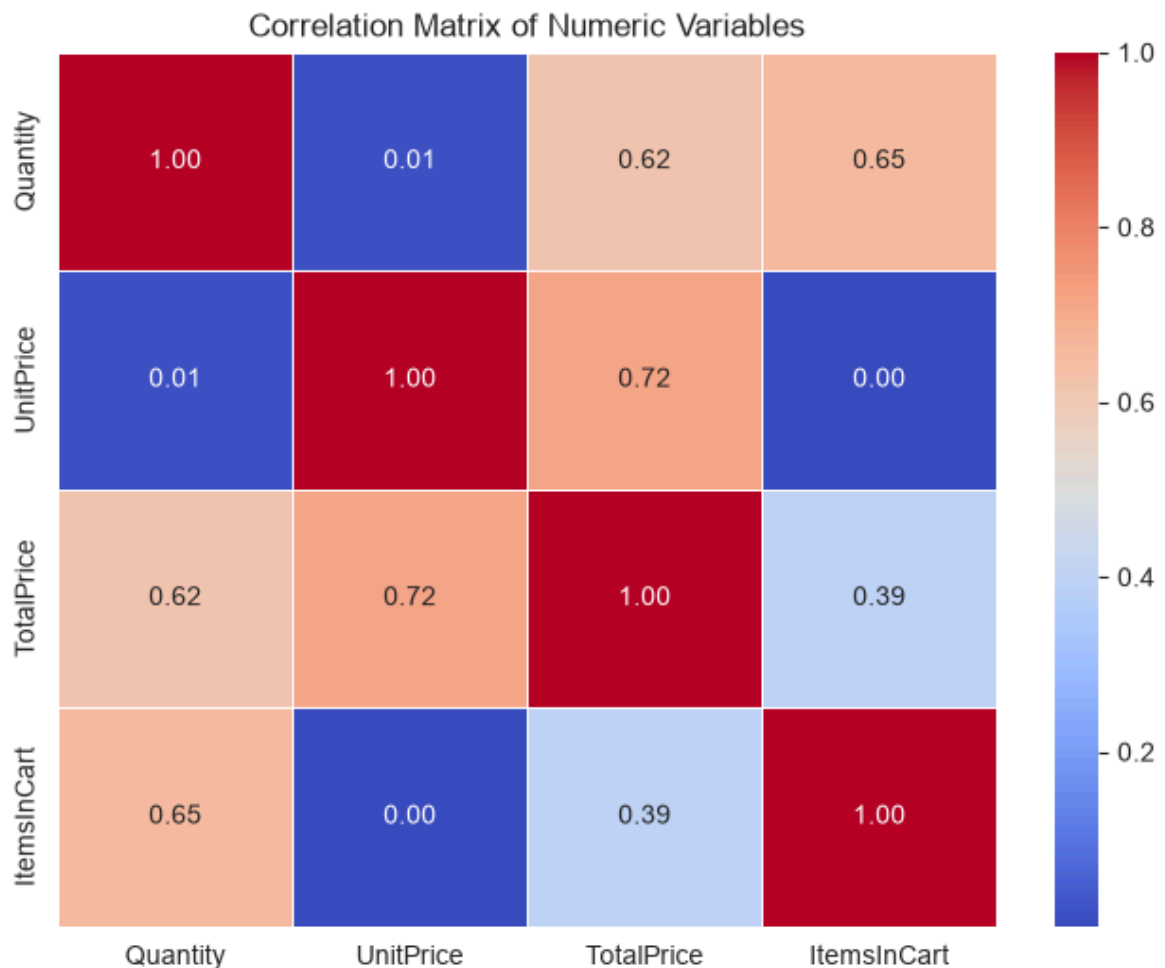
```
# Calculate correlation matrix
correlation_matrix = numeric_df.corr()

print("=== Correlation Matrix ===\n")
print(correlation_matrix)
```

=== Correlation Matrix ===

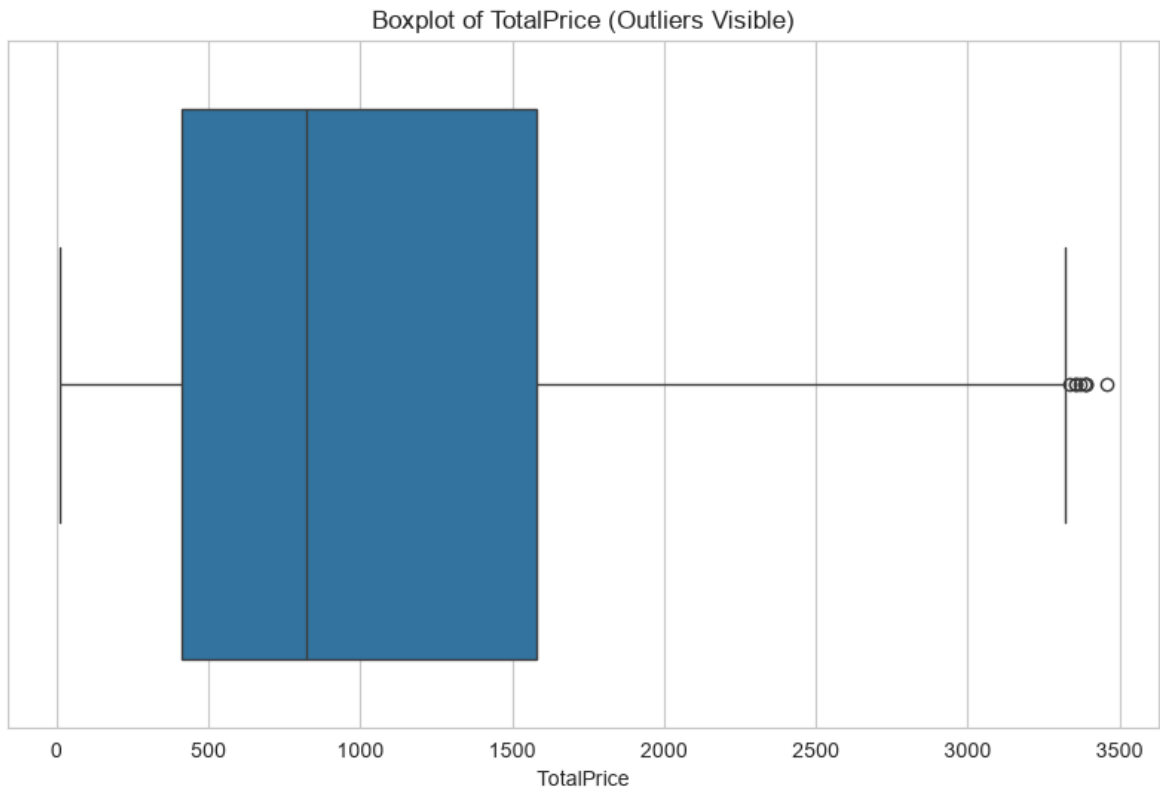
	Quantity	UnitPrice	TotalPrice	ItemsInCart
Quantity	1.000000	0.014553	0.615251	0.650061
UnitPrice	0.014553	1.000000	0.717081	0.000602
TotalPrice	0.615251	0.717081	1.000000	0.392540
ItemsInCart	0.650061	0.000602	0.392540	1.000000

```
In [12]: # Plot correlation heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt='.2f', l
plt.title('Correlation Matrix of Numeric Variables')
plt.show()
```

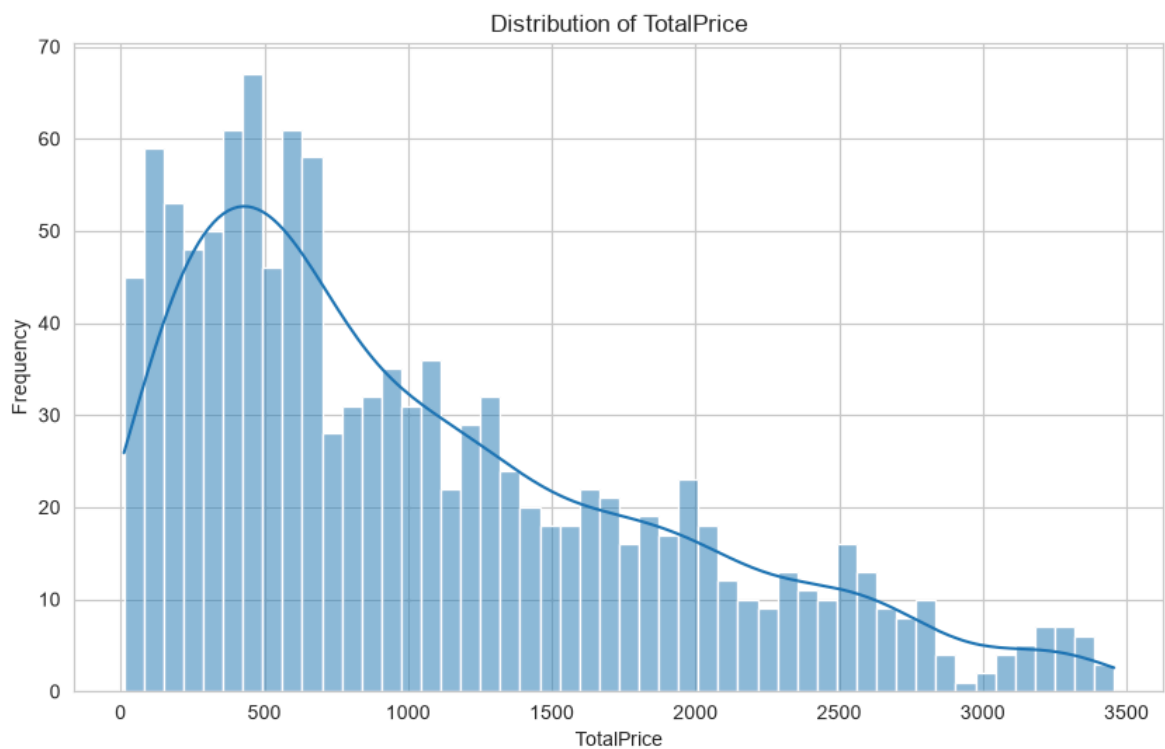


```
In [13]: # Boxplot for TotalPrice to visualize outliers
plt.figure(figsize=(10, 6))
sns.boxplot(x=df['TotalPrice'])
plt.title('Boxplot of TotalPrice (Outliers Visible)')
plt.xlabel('TotalPrice')

plt.show()
```



```
In [14]: # Histogram with KDE for TotalPrice
plt.figure(figsize=(10, 6))
sns.histplot(df['TotalPrice'], bins=50, kde=True)
plt.title('Distribution of TotalPrice')
plt.xlabel('TotalPrice')
plt.ylabel('Frequency')
plt.show()
```



```
In [18]: print("=" * 60)
print("                KEY OBSERVATIONS: EDA SUMMARY")
print("=" * 60)
```

```

print("\n1. DATASET OVERVIEW:")
print(f"    - Total orders: {df.shape[0]}")
print(f"    - Total columns: {df.shape[1]}")

print("\n2. CENTRAL TENDENCY:")
for col in ['TotalPrice', 'Quantity']:
    print(f"    - {col}: Mean = {df[col].mean():.2f}, Median = {df[col].me

print("\n3. TOP PRODUCTS BY REVENUE:")
for product, revenue in product_revenue.head(3).items():
    print(f"    - {product}: ${revenue:,.2f}")

print("\n4. ORDER STATUS DISTRIBUTION:")
for status, count in df['OrderStatus'].value_counts().head(3).items():
    print(f"    - {status}: {count} orders")

print("\n5. OUTLIERS:")
print(f"    - Total outliers in TotalPrice: {len(outliers)} ({len(outliers

print("\n6. CORRELATIONS:")
print(f"    - TotalPrice vs Quantity: {correlation_matrix.loc['TotalPrice'
print(f"    - TotalPrice vs UnitPrice: {correlation_matrix.loc['TotalPrice

print("\n" + "=" * 60)
print("EDA Completed Successfully!")

```

=====

KEY OBSERVATIONS: EDA SUMMARY

=====

1. DATASET OVERVIEW:
 - Total orders: 1200
 - Total columns: 15
2. CENTRAL TENDENCY:
 - TotalPrice: Mean = 1053.97, Median = 823.62
 - Quantity: Mean = 2.95, Median = 3.00
3. TOP PRODUCTS BY REVENUE:
 - Chair: \$195,620.11
 - Printer: \$195,612.61
 - Laptop: \$192,126.56
4. ORDER STATUS DISTRIBUTION:
 - Cancelled: 250 orders
 - Returned: 247 orders
 - Pending: 237 orders
5. OUTLIERS:
 - Total outliers in TotalPrice: 8 (0.67%)
6. CORRELATIONS:
 - TotalPrice vs Quantity: 0.62
 - TotalPrice vs UnitPrice: 0.72

=====

EDA Completed Successfully!

In []: